Senior Engineer Code Test

Overview

This test is designed to assess your expertise in:

- The tech stack: Node.js, NestJS, MongoDB, Kafka, Kafka, Elasticsearch.
- Architectural principles: Domain-Driven Design (DDD), Event-Driven Architecture (EDA), Multi-Tenant Systems.
- Software engineering skills: Data structures, algorithms, SOLID principles, and design patterns.

You are expected to write production-quality code and follow best practices.

Test Format

- **Duration:** 4–5 hours
- **Submission:** Code repository (GitHub/Bitbucket) + README with architecture decisions.

Problem: Build RESTful APIs for message management.

Functional Requirements

1. **Tech Stack:**
    - Use NestJS for building RESTful APIs.
    - MongoDB as the primary data store.
    - Kafka for message brokering.
    - Elasticsearch for search functionality.
2. **API Endpoints:**
    - POST /api/messages to create a new message.

        - Validate input for required fields (conversationId, content).
        - Store the message in MongoDB as the primary data store.
        - Publish message events to the broker.
    - GET /api/conversations/:conversationId/messages to retrieve messages for a conversation.
        - Support pagination and sorting.
    - GET /api/conversations/:conversationId/messages/search?q=term for searching messages.
        - Implement full-text search using Elasticsearch.
        - Define and configure appropriate Elasticsearch mappings to ensure efficient search performance.
3. **Data Model & Indexing:**
    - **Message Type Example:**

```
type Message = {
  id: string;
  conversationId: string;
  senderId: string;
  content: string;
  timestamp: Date;
  metadata?: Record<string, any>;
};
```

4. Message schema with fields: id, conversationId, content, timestamp.
    - Ensure efficient indexing for search and retrieval.
    - Define appropriate MongoDB indexes for optimized query performance.
5. **Event Handling & Messaging:**
    - Publish message creation events to Kafka.
    - Implement a subscriber to process and index messages into Elasticsearch.
    - Design Kafka topics, partitions, and consumer groups for scalability and fault tolerance.
6. **Code Quality:**
    - Follow SOLID principles.
    - Write both unit tests and API integration tests.
    - Provide a comprehensive README with setup instructions and architecture decisions.
7. **Security:**
    - Validate and sanitize inputs.
    - Implement basic authentication and authorization (optional).
8. **Performance:**
    - Optimize database queries.
    - Implement caching (optional).

Non-Functional Requirements

Performance & Scalability:

- Efficient MongoDB retrieval with proper indexing.
- Optimize database queries and consider caching.

Reliability & Security:

- Ensure message delivery guarantees.
- Validate and sanitize inputs.
- Implement basic authentication and authorization.

Code Quality & Best Practices:

- Follow SOLID principles.
- Adhere to DDD practices.
- Implement both unit tests and integration tests covering key functionalities and edge cases.
- Handle multi-tenancy effectively.
- Optimize data structures.

Please follow the instructions carefully and submit your solutions within the given timeframe.